INF2008 Machine Learning
Speaker Verification Project
P1 Group 19

USER MANUAL

Table of Contents

Project Overview

This document provides an overview of the **INF2008 Machine Learning** project, developed by **Team 19 P1**, which consists of:

- NG WEI SHEN, JACKSON
- MUNIR RUDY HERMAN
- NURTIARA TANIA RAHIM
- NICHOLAS PHOON KAI JIN
- JOLIN TAN

Project Description

In this project, we focused on solving a **Speaker Verification** task. The goal of the task is to determine whether a given audio sample belongs to a specific speaker. To achieve this, we employed a **binary classifier**.

Benchmark

Our approach is compared against a benchmark established by **Prof. Xiao Xiao** in their paper, where they reported an **Equal Error Rate (EER) of 4.59%** on their dataset. This was achieved using the **ECAPA-TDNN** deep learning model, which is considered a state-of-the-art method for this task.

Feature Engineering

Our team implemented three different feature engineering methods to enhance the model's performance:

1. **Product and Difference between embeddings**
2. **Manhattan distance between embeddings**
3. **Clustering with HDBSCAN**

Models Tested

We tested three machine learning models in our experiments:

- **Logistic Regression**
- **XGBoost**
- **Support Vector Machine (SVM)**

Among these models, the **Support Vector Machine (SVM)** achieved the best **Equal Error Rate (EER) of 3.53%**, outperforming the benchmark set by Prof. Xiao Xiao.

Demo

Additionally, to provide a clearer understanding of our task and its implementation, we developed a **demo**. This demo serves as a practical tool to showcase how the speaker verification system works in action.

Directory Structure

```
inf2008_machine_learning/
├── demo/                          # Contains the code for both the frontend and backend of the demo
├── extraction_models/             # Contains Jupyter notebooks that were used to use the model and extract embeddings from
the wav file
│
├── test_set/                      # Dataset used for testing the model (Same test set as benchmark for fair comparison)
│   ├── data/                       # Raw test data (Speaker embeddings, combined speaker embeddings and trial embeddings)
│   ├── feature_vector/             # Extracted feature vectors for test data
│
├── training_set/                   # Dataset used for training the model (Distinct from test set to prevent data leakage)
│   ├── data/
│   ├── feature_vector/
│
├── validation_set/                 # Dataset used for hyperparameter tuning.
│   ├── data/
│   ├── feature_vector/
│
├── feature_engineering.ipynb       # Jupyter notebook for feature extraction and preprocessing
├── model_training_and_eval.ipynb   # Jupyter notebook for training and evaluating the model
├── utils.py                        # Utility functions used in both notebooks above.
```

Training and Evaluating Models

Follow these steps to set up and run the feature engineering and model training/evaluation processes.

Step 1: Install Dependencies

Before proceeding, make sure you have all the required dependencies installed. To do this, run the following command: `pip install -r requirements.txt`

This will install all the necessary libraries and tools for the project.

Step 2: Run Feature Engineering (Optional)

If you want to re-generate the features for the training, validation, and test datasets, follow these steps. However, you may skip this steps as the features have already been created in feature_vector folder of each dataset.:

1. Open the `feature_engineering.ipynb` notebook.

2. Run all the cells in the notebook. This process will:

    - Take the embeddings from the **data** folder in each dataset (training, validation, and test).
    - Combine the Speaker embeddings.
    - Compute the **Product & Difference** between the embeddings.
    - Compute the **Manhattan Distance** between the embeddings.
    - Perform **Clustering** using HDBSCAN.

3. After the process is complete, the generated features will be saved in the respective **feature_vector** folders:

    - **features.npy**: Contains the X data (features).
    - **label.npy**: Contains the corresponding y data (labels).

Step 4: Run Model Training and Evaluation

To train and evaluate the models:

1. Open the `model_training_and_eval.ipynb` notebook.
2. Run all the cells in the notebook. This will:
    - Load the previously generated features from the **feature_vector** folders.
    - Train and evaluate the models (Logistic Regression, XGBoost, SVM) on the data.
    - Compute and display evaluation metrics, such as the **Equal Error Rate (EER)** and DET curves.

*Note: Please be aware that the **EER** may vary across different machines, even when using the same random state. We believe that this inconsistency may arise from random states that are present at lower levels in the model training process, despite setting a fixed random state in the notebook.*

By following these steps, you should be able to run the full process of feature engineering, model training, and evaluation smoothly.

Setting Up Demo

Follow these steps to set up and run the demo:

1. Install Required Packages

Run the following command to install all dependencies from the `requirements.txt` file:

```
pip install -r requirements.txt
```

2. Install `ffmpeg`

Navigate to the backend directory and install `ffmpeg`:

```
cd demo/src/backend
```

```
sudo apt-get update
```

```
sudo apt-get install ffmpeg
```

3. Install the TTS Package

Install the TTS package directly from GitHub:

```
pip install git+https://github.com/munir2200963/TTS.git
```

If Step 3 Fails, Follow These Steps:

3a) Clone the TTS Repository

```
git clone https://github.com/munir2200963/TTS.git
```

For some reason, the TTS repository is nested within another TTS folder. Follow these steps to fix it:

- **Drag the nested TTS folder out** and rename it to `TTS1`.
- Delete the old `TTS` folder.
- Rename `TTS1` back to `TTS`.

4. Install Additional Dependencies

Install the necessary libraries:

```
sudo apt-get install -y espeak libsndfile1
```

5. Download TTS Model

Download the pre-trained TTS model:

```
wget
https://coqui.gateway.scarf.sh/v0.7.0_models/tts_models--en--blizzard2013--ca
pacitron-t2-c50.zip
```

6. Unzip the Model

Extract the contents of the downloaded `.zip` file:

```
unzip '*.zip'
```

7. Downgrade Dependencies (If Required)

You might need to downgrade `torchaudio` and `torch` to fix the weight loading issue. Run the following commands:

```
pip install torchaudio==2.5.0

pip install torch==2.5.0
```

If the above doesn't work, you can modify the `TTS/utils/io.py` file:

- Go to **Line 54** and change:

```
return torch.load(f, map_location=map_location, **kwargs)
```

To:

```
return torch.load(f, map_location=map_location, weights_only=False, **kwargs)
```

8. Start the Backend Server

Run the backend server:

```
python3 server.py
```

9. Start the Frontend

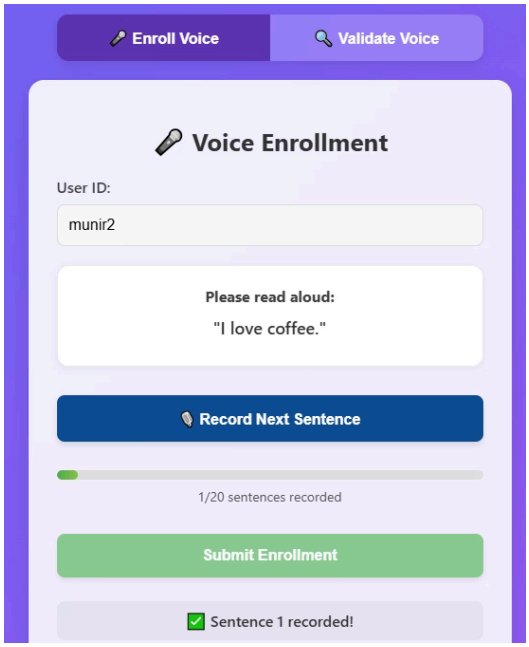
In a **new terminal**, navigate back to the `demo` directory and run:

```
cd demo
npm install
npm start
```

Using the Demo

Once you have the demo running, open your web browser and navigate to `localhost:3000` to access the demo site.

Step 1: Record Enroll Audio



Snippet of Enroll Voice Tab

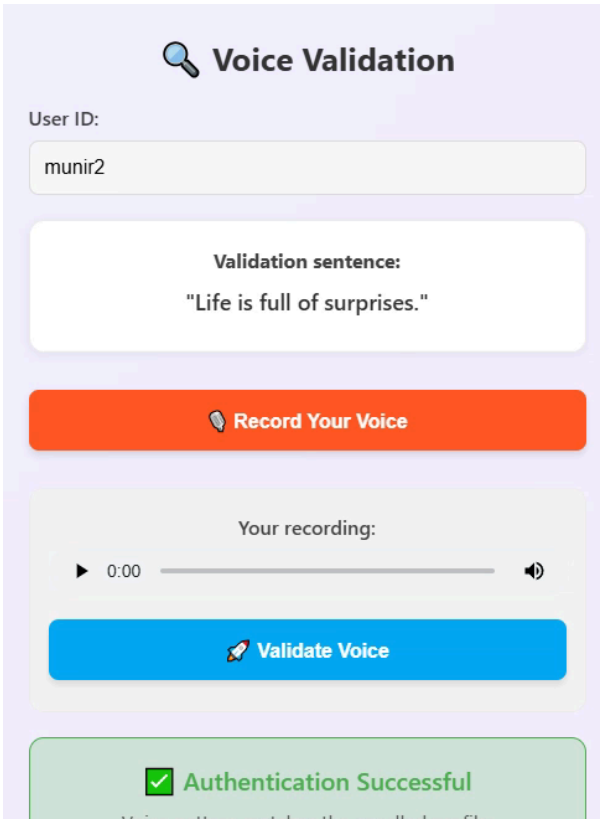
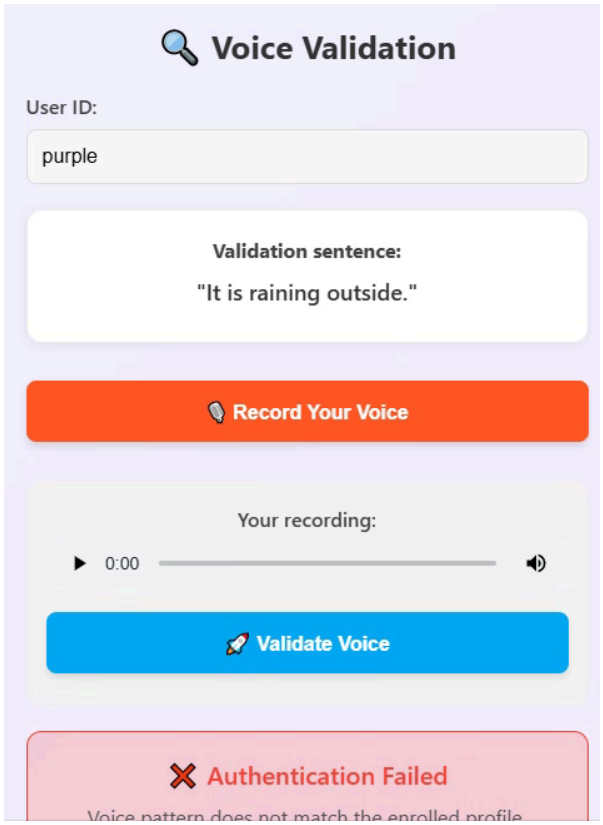
1. Go to the **Enroll Voice** section of the demo.
2. Enter your name to create a new profile.
3. You will then be prompted to record **20 sentences** of your own voice. This will be used to create your unique voiceprint.
4. After you have recorded all 20 sentences, click **Submit Enrollment** to save your voiceprint.

Step 2: Validate Voice

To test the speaker verification model:

1. Go to the **Validate Voice** tab in the demo.
2. Enter your **user ID** (the same one used for enrollment).
3. You will be prompted to **record** yourself saying a specific sentence. Click on the button labeled **Record your voice** and speak the sentence.
4. After recording, click **Validate Voice**. The model will take around **5 seconds** to process the input.
5. Once done, the result will be displayed as either **Authentication Successful** or **Authentication Failed**, depending on whether it recognizes your voice or not.

This process allows you to test if the speaker verification model correctly identifies your voice and matches it to the profile you created.

	
Snippet of Validate VoiceTab for Successful Authentication	Snippet of Validate Voice tab for Failed Authentication